

Image Processing Pipeline

MNIST PNG Dataset - CS.383 Group Project

1. Project Overview

This project implements a four-stage image processing pipeline on the MNIST handwritten digit dataset (PNG format). The dataset is downloaded automatically via kagglehub. Each stage builds on the previous one: dataset collection, preprocessing, filtering, and edge detection with segmentation. The project is part of CS.383.

2. Project Structure

File	Description
project_pipeline.py	Full pipeline - runs all 4 stages end-to-end
dataset_collection.py	Stage 1 - Dataset exploration and visualization
preprocessing.py	Stage 2 - Resize, grayscale, normalize, invert
filtering.py	Stage 3 - Gaussian blur, median filter, sharpening
edge_detection.py	Stage 4 - Sobel, Canny, Otsu segmentation

3. Pipeline Stages

Stage 1 - Dataset Collection ([dataset_collection.py](#))

- Downloads dataset automatically using kagglehub
- Scans digit subfolders 0-9 and reports image counts
- Displays one sample per digit in a 2x5 matplotlib grid

Stage 2 - Preprocessing ([preprocessing.py](#))

- Resizes images to 28x28 pixels with anti-aliasing
- Converts RGB to grayscale via `skimage.color.rgb2gray`
- Normalizes pixel values to [0, 1]
- Applies intensity inversion: $\text{image_inv} = 1.0 - \text{image_norm}$

Stage 3 - Image Filtering ([filtering.py](#))

- Gaussian Blur - smooths noise, Gaussian kernel ($\text{sigma}=1$)
- Median Filter - removes salt-and-pepper noise, disk element ($\text{radius}=1$)
- Sharpening - 3x3 Laplacian-based kernel via `cv2.filter2D`

Stage 4 - Edge Detection & Segmentation ([edge_detection.py](#))

- Sobel - gradient magnitude in X and Y directions
- Canny - multi-stage edge detector (thresholds: 50 / 150)
- Otsu Threshold - automatic binary segmentation (`cv2.THRESH_OTSU`)

4. Dataset

The dataset is downloaded automatically on first run using kagglehub. No manual download is needed. The cached copy is reused on subsequent runs.

```
import kagglehub

path = kagglehub.dataset_download("ben519/mnist-as-png")
print("Path to dataset files:", path)
```

The dataset (ben519/mnist-as-png on Kaggle) contains PNG images of handwritten digits. Images are stored under mnist-png/train/0-9/ inside the downloaded path. The scripts resolve this path automatically.

5. Requirements

```
pip install kagglehub scikit-image opencv-python matplotlib numpy
```

Library	Used For
kagglehub	Automatic dataset download from Kaggle
scikit-image	I/O, resize, color conversion, Gaussian/median filtering
opencv-python	Sharpening, Sobel, Canny, Otsu thresholding
matplotlib	Visualization of results at each stage
numpy	Array operations and type conversions

6. Usage

Linux

```
pip install kagglehub scikit-image opencv-python matplotlib numpy

python3 project_pipeline.py          # full pipeline (recommended)

python3 dataset_collection.py        # stage 1 only
python3 preprocessing.py             # stage 2 only
python3 filtering.py                 # stage 3 only
python3 edge_detection.py            # stage 4 only
```

Windows

Install Python 3 from python.org (check 'Add Python to PATH'), then:

```
pip install kagglehub scikit-image opencv-python matplotlib numpy

python project_pipeline.py          # full pipeline (recommended)

python dataset_collection.py        # stage 1 only
```

```
python preprocessing.py          # stage 2 only
python filtering.py              # stage 3 only
python edge_detection.py        # stage 4 only
```

7. Expected Output

Stage	Matplotlib Figure
Dataset Collection	2x5 grid - one sample digit image per class (0-9)
Preprocessing	4 panels: Original / Resized / Normalized / Inverted
Filtering	4 panels: Grayscale / Gaussian / Median / Sharpened
Edge Detection	4 panels: Grayscale / Sobel / Canny / Otsu

8. How to Test the Project

Follow these steps to verify the project works correctly:

- Step 1 - Install all dependencies (see Section 5)
- Step 2 - Run: `python3 project_pipeline.py` (Linux) or `python project_pipeline.py` (Windows)
- Step 3 - The dataset downloads automatically; the terminal prints its local path
- Step 4 - A window appears: 2x5 grid of sample digit images (Stage 1)
- Step 5 - Close the window; next window shows 4 preprocessing panels (Stage 2)
- Step 6 - Close the window; next window shows 4 filtering panels (Stage 3)
- Step 7 - Close the window; next window shows 4 edge detection panels (Stage 4)
- Step 8 - Terminal prints: Full pipeline complete!

To test individual stages, run each script separately. Each script downloads the dataset automatically and shows its own output figure.